API Endpoints Reference - JNX-OS v2 + Qryx

Table of Contents

Overview

Base URLs

- **Production**: `https://www.jnxlabs.ai`
- **Development**: `http://localhost:3000`

Authentication

Most endpoints require authentication via **Clerk**.

Client-Side:

```
import { useAuth } from '@clerk/nextjs';
const { getToken } = useAuth();
const token = await getToken();
```

Server-Side:

```
import { auth } from '@clerk/nextjs/server';
const { userId } = auth();
```

Common Headers

```
Content-Type: application/json
Authorization: Bearer <clerk_token>
```

Rate Limiting

- **General**: 100 requests/minute
- **Auth**: 10 requests/minute
- **Chat**: 30 requests/minute

Authentication Endpoints

🔒 Deprecated Auth Endpoints

The following endpoints are **DEPRECATED** and return `410 Gone`:

- `POST /api/auth/signup`
- `POST /api/auth/login`
- `GET /api/auth/user`
- `POST /api/auth/google`

Reason: JNX-OS now uses **Clerk** for all authentication.

Alternatives:
- Signup: Navigate to `/signup` (Clerk hosted UI)
- Login: Navigate to `/login` (Clerk hosted UI)
- Get User: Use `currentUser()` from `@clerk/nextjs/server`
- Google OAuth: Handled automatically by Clerk

Qryx Installation Endpoints

1. Initiate Installation

Endpoint: `GET /api/qryx/install`

Description: Starts the Qryx installation flow. Creates an encrypted shop session and redirects to login.

Query Parameters:

Parameter	Type	Required	Description
`shop`	string	✅ Yes	Shopify shop domain (my-shop.myshopify.com)

Example Request:

```
GET /api/qryx/install?shop=shopbotv3.myshopify.com HTTP/1.1
Host: www.jnxlabs.ai
```

Response:

```
HTTP/1.1 302 Found
Location: /login
Set-Cookie: shop_session=eyJhbGc...; Path=/; HttpOnly; Secure; SameSite=Lax; Max-Age=1800
```

Success:
- Status: `302 Found`

- Redirects to `/login`
- Sets `shop_session` cookie (30-minute expiry)

Errors:

```
{
  "error": "Missing shop parameter",
  "code": "MISSING_SHOP"
}
```

Status Codes:
- `302` : Success, redirected to login
- `400` : Missing or invalid shop parameter
- `500` : Server error (session encryption failed)

---

2. OAuth Callback

Endpoint: `GET /api/qryx/callback`

Description: Handles Shopify OAuth callback after successful payment. Exchanges authorization code for access token.

Query Parameters:

Parameter	Type	Required	Description
`code`	string	✅ Yes	OAuth authorization code
`shop`	string	✅ Yes	Shopify shop domain
`hmac`	string	✅ Yes	HMAC signature for verification
`state`	string	❌ No	Optional state parameter

Example Request:

```
GET /api/qryx/callback?code=abc123&shop=shopbotv3.myshopify.com&hmac=xyz789 HTTP/1.1
Host: www.jnxlabs.ai
```

Response:

```
HTTP/1.1 302 Found
Location: /app/products/qryx
```

Success:
- Status: `302 Found`
- Redirects to `/app/products/qryx`

- Creates `qryx_shops` database record
- Stores encrypted access token

Errors:

```
{
  "error": "Invalid HMAC signature",
  "code": "INVALID_HMAC"
}
```

Status Codes:
- `302` : Success, redirected to dashboard
- `400` : Invalid parameters or HMAC
- `401` : No active subscription found
- `500` : Server error

Qryx Core Endpoints

3. Get Qryx Configuration

Endpoint: `GET /api/qryx/config`

Description: Retrieves Qryx chatbot configuration and subscription details for a shop.

Authentication: Required (Clerk)

Query Parameters:

Parameter	Type	Required	Description
`shop`	string	✅ Yes	Shopify shop domain

Example Request:

```
GET /api/qryx/config?shop=shopbotv3.myshopify.com HTTP/1.1
Host: www.jnxlabs.ai
Authorization: Bearer <clerk_token>
```

Example Response:

```
{
  "shop": "shopbotv3.myshopify.com",
  "chatbotConfig": {
    "enabled": true,
    "theme": {
      "primaryColor": "#3b82f6",
      "position": "bottom-right",
      "greeting": "Hi! How can I help you today?"
    },
    "behavior": {
      "autoOpen": false,
      "soundEnabled": true,
      "typingIndicator": true
    }
  },
  "subscription": {
    "plan": "starter",
    "status": "active",
    "conversationLimit": 500,
    "conversationCount": 147,
    "currentPeriodEnd": "2025-01-28T12:00:00Z"
  }
}
```


Status Codes:
- `200` : Success
- `401` : Unauthorized (not authenticated)
- `403` : Forbidden (shop doesn’t belong to user)
- `404` : Shop not found
- `500` : Server error

---

4. Update Qryx Configuration

Endpoint: `POST /api/qryx/config`

Description: Updates chatbot configuration for a shop.

Authentication: Required (Clerk)

Request Body:


```
{
  "shop": "shopbotv3.myshopify.com",
  "config": {
    "theme": {
      "primaryColor": "#10b981",
      "position": "bottom-left",
      "greeting": "Welcome! Need help finding something?"
    },
    "behavior": {
      "autoOpen": true,
      "soundEnabled": false,
      "typingIndicator": true
    }
  }
}
```

Example Response:

```
{
  "success": true,
  "message": "Configuration updated successfully",
  "config": {
    "theme": {...},
    "behavior": {...}
  }
}
```

Status Codes:

- `200` : Success
- `400` : Invalid configuration
- `401` : Unauthorized
- `403` : Forbidden
- `500` : Server error

---

5. Chat Endpoint

Endpoint: `POST /api/qryx/chat`

Description: Processes a chat message and returns AI-generated response using Gemini 2.0 Flash.

Authentication: Shop-based (via shop domain)

Request Body:

```
{
  "shop": "shopbotv3.myshopify.com",
  "message": "Do you have any blue running shoes?",
  "conversationId": "conv_abc123xyz",
  "context": {
    "customerId": "cust_456",
    "sessionId": "sess_789"
  }
}
```

Example Response:

```
{
  "response": "Yes! We have several blue running shoes. Our most popular is the Nike Air Zoom Pegasus in Royal Blue, available in sizes 7-12. Would you like to see more details?",
  "conversationId": "conv_abc123xyz",
  "suggestions": [
    "Show me the shoes",
    "What sizes are available?",
    "Tell me about other colors"
  ],
  "metadata": {
    "model": "gemini-2.0-flash",
    "tokens": 234,
    "responseTime": 1.2
  }
}
```


Rate Limiting: 30 requests/minute per shop

Status Codes:
- `200` : Success
- `400` : Invalid request body
- `403` : Subscription inactive or limit exceeded
- `404` : Shop not found
- `429` : Rate limit exceeded
- `500` : Server error (Gemini API failure)

Stripe Billing Endpoints

6. Create Checkout Session

Endpoint: `POST /api/stripe/checkout`

Description: Creates a Stripe Checkout Session for Qryx subscription.

Authentication: Required (Clerk)

Request Body:


```
{
  "planId": "starter",
  "shop": "shopbotv3.myshopify.com"
}
```


Valid Plan IDs:
- `starter` - $29/month, 500 conversations
- `professional` - $79/month, 2,000 conversations
- `business` - $199/month, 5,000 conversations

Example Response:

```
{
  "sessionId": "cs_test_a1B2c3D4e5F6g7H8i9J0k1L2m3N4o5P6q7R8s9T0u1V2w3X4y5Z6",
  "url": "https://checkout.stripe.com/c/pay/cs_test_a1B2c3..."
}
```

Flow:
1. Validates shop session (must exist and not be expired)
2. Gets Clerk user ID
3. Creates Stripe Checkout Session
4. Attaches metadata: `{ shop: "shopbotv3.myshopify.com", clerkUserId: "user_123" }`
5. Returns checkout URL

Status Codes:
- `200` : Success
- `400` : Invalid plan ID or missing shop
- `401` : Unauthorized
- `404` : Shop session expired
- `500` : Server error (Stripe API failure)

---

7. Stripe Webhook

Endpoint: `POST /api/stripe/webhook`

Description: Handles Stripe webhook events for subscription management.

Authentication: Stripe Signature Verification

Headers Required:

```
stripe-signature: t=1234567890,v1=abc123...
```

Events Handled:

7.1 `checkout.session.completed`

Triggered: After successful payment

Action:
- Creates `billing_subscriptions` record
- Stores Stripe customer ID and subscription ID
- Sets status to `active`
- Stores shop domain from metadata

Example Event Data:

```
{
  "id": "evt_abc123",
  "type": "checkout.session.completed",
  "data": {
    "object": {
      "id": "cs_test_abc123",
      "customer": "cus_abc123",
      "subscription": "sub_abc123",
      "metadata": {
        "shop": "shopbotv3.myshopify.com",
        "clerkUserId": "user_abc123"
      }
    }
  }
}
```

7.2 `customer.subscription.updated`

Triggered: When subscription status changes

Action:
- Updates subscription status
- Updates billing period dates
- Updates `cancel_at_period_end` flag

7.3 `customer.subscription.deleted`

Triggered: When subscription is canceled

Action:
- Sets status to `canceled`
- Sets `canceled_at` timestamp
- Keeps historical record (soft delete)

7.4 `invoice.payment_succeeded`

Triggered: On successful recurring payment

Action:
- Renews subscription period
- Resets conversation counter
- Ensures status is `active`

7.5 `invoice.payment_failed`

Triggered: When payment fails

Action:
- Sets status to `past_due`
- Logs failure event
- Stripe handles retry logic

Response:

```
{
  "received": true
}
```

Status Codes:
- `200` : Success
- `400` : Invalid signature
- `500` : Server error (database failure)

---

8. Get Subscription Status

Endpoint: `GET /api/stripe/subscription`

Description: Retrieves current subscription status for authenticated user.

Authentication: Required (Clerk)

Query Parameters:

Parameter	Type	Required	Description
`shop`	string	❌ No	Filter by specific shop

Example Request:

```
GET /api/stripe/subscription?shop=shopbotv3.myshopify.com HTTP/1.1
Host: www.jnxlabs.ai
Authorization: Bearer <clerk_token>
```

Example Response:

```
{
  "subscriptions": [
    {
      "id": "uuid-123",
      "shop": "shopbotv3.myshopify.com",
      "plan": "starter",
      "status": "active",
      "currentPeriodStart": "2024-12-29T12:00:00Z",
      "currentPeriodEnd": "2025-01-29T12:00:00Z",
      "cancelAtPeriodEnd": false,
      "conversationLimit": 500,
      "conversationCount": 147
    }
  ]
}
```

Status Codes:
- `200` : Success
- `401` : Unauthorized
- `404` : No subscriptions found
- `500` : Server error

---

9. Cancel Subscription

Endpoint: `POST /api/stripe/subscription/cancel`

Description: Cancels a subscription at the end of the billing period.

Authentication: Required (Clerk)

Request Body:

```
{
  "subscriptionId": "sub_abc123",
  "immediately": false
}
```

Parameters:
- `immediately`: If `true`, cancels immediately. If `false`, cancels at period end.

Example Response:

```
{
  "success": true,
  "message": "Subscription will be canceled at the end of the billing period",
  "cancelAt": "2025-01-29T12:00:00Z"
}
```

Status Codes:
- `200`: Success
- `400`: Invalid subscription ID
- `401`: Unauthorized
- `403`: Subscription doesn’t belong to user
- `500`: Server error

System & Admin Endpoints

10. System Health Check

Endpoint: `GET /api/system/health`

Description: Returns system health metrics and status of all integrations.

Authentication: Optional (more details if authenticated as admin)

Example Request:

```
GET /api/system/health HTTP/1.1
Host: www.jnxlabs.ai
```

Example Response:

```
{
  "status": "operational",
  "timestamp": "2025-12-29T12:00:00Z",
  "services": {
    "clerk": {
      "status": "operational",
      "authenticated": true,
      "userId": "user_abc123"
    },
    "supabase": {
      "status": "operational",
      "latency": 45
    },
    "stripe": {
      "status": "operational",
      "mode": "live"
    },
    "gemini": {
      "status": "operational",
      "model": "gemini-2.0-flash"
    },
    "shopify": {
      "status": "operational",
      "connectedShops": 12
    }
  },
  "metrics": {
    "activeSessions": 48,
    "activeSubscriptions": 12,
    "totalConversations24h": 1247
  }
}
```


Status Codes:
- `200` : All systems operational
- `503` : One or more services degraded

---

11. Clerk Webhook

Endpoint: `POST /api/webhooks/clerk`

Description: Handles Clerk webhook events for user and organization sync.

Authentication: Svix Signature Verification

Headers Required:

```
svix-id: msg_abc123
svix-timestamp: 1234567890
svix-signature: v1,abc123...
```

Events Handled:

- `user.created` - Creates user in database
- `user.updated` - Updates user information
- `organization.created` - Creates organization

- `organization.updated` - Updates organization
- `organizationMembership.created` - Links user to org

Example Event:

```
{
  "type": "user.created",
  "data": {
    "id": "user_abc123",
    "email_addresses": [
      {
        "email_address": "user@example.com"
      }
    ],
    "first_name": "John",
    "last_name": "Doe"
  }
}
```

Response:

```
{
  "success": true
}
```

Status Codes:
- `200` : Success
- `400` : Invalid signature
- `500` : Server error

Widget Endpoints

12. Widget Script

Endpoint: `GET /widget/qryx`

Description: Returns JavaScript widget for embedding Qryx chatbot in Shopify stores.

Authentication: None (public endpoint)

Query Parameters:

Parameter	Type	Required	Description
`shop`	string	✅ Yes	Shopify shop domain

Example Request:

```
<script src="https://www.jnxlabs.ai/widget/qryx?shop=shopbotv3.myshopify.com"></
script>
```

Response:

```
(function() {
  // Widget initialization code
  const shop = 'shopbotv3.myshopify.com';
  const config = { /* fetched from API */ };

  // Render chatbot interface
  // Handle user messages
  // Display AI responses
})();
```

Status Codes:
- `200` : Success
- `403` : Shop subscription inactive
- `404` : Shop not found

---

13. Widget Configuration

Endpoint: `GET /api/widget/qryx`

Description: Returns widget configuration for a specific shop (used by widget script).

Authentication: None (validated by shop domain)

Query Parameters:

Parameter	Type	Required	Description
`shop`	string	✅ Yes	Shopify shop domain

Example Request:

```
GET /api/widget/qryx?shop=shopbotv3.myshopify.com HTTP/1.1
Host: www.jnxlabs.ai
```

Example Response:

```
{
  "enabled": true,
  "theme": {
    "primaryColor": "#3b82f6",
    "position": "bottom-right",
    "greeting": "Hi! How can I help you today?",
    "avatar": "https://img.freepik.com/free-vector/chatbot-chat-message-vector-
art_78370-4104.jpg?semt=ais_hybrid&w=740&q=80"
  },
  "behavior": {
    "autoOpen": false,
    "soundEnabled": true,
    "typingIndicator": true,
    "showPoweredBy": true
  },
  "apiEndpoint": "https://www.jnxlabs.ai/api/qryx/chat"
}
```

Status Codes:
- `200` : Success
- `403` : Subscription inactive or limit exceeded
- `404` : Shop not found

Error Codes

Standard Error Response Format

```
{
  "error": "Human-readable error message",
  "code": "MACHINE_READABLE_CODE",
  "details": {
    "field": "Additional context"
  },
  "timestamp": "2025-12-29T12:00:00Z"
}
```

Common Error Codes

Code	HTTP Status	Description
`UNAUTHORIZED`	401	Missing or invalid authentication
`FORBIDDEN`	403	Insufficient permissions
`NOT_FOUND`	404	Resource not found
`INVALID_REQUEST`	400	Malformed request body
`MISSING_PARAMETER`	400	Required parameter missing
`INVALID_SHOP`	400	Invalid shop domain format
`SESSION_EXPIRED`	401	Shop session expired
`SUBSCRIPTION_INACTIVE`	403	Subscription not active
`LIMIT_EXCEEDED`	403	Conversation limit reached
`RATE_LIMIT_EXCEEDED`	429	Too many requests
`WEBHOOK_SIGNATURE_INVALID`	400	Invalid webhook signature
`STRIPE_API_ERROR`	500	Stripe API failure
`DATABASE_ERROR`	500	Database operation failed
`GEMINI_API_ERROR`	500	AI model API failure
`SHOPIFY_API_ERROR`	500	Shopify API failure

Error Handling Best Practices

Client-Side:

```
try {
  const response = await fetch('/api/qryx/chat', {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${token}`
    },
    body: JSON.stringify({ shop, message })
  });

  if (!response.ok) {
    const error = await response.json();

    switch (error.code) {
      case 'SESSION_EXPIRED':
        // Redirect to re-install
        window.location.href = `/api/qryx/install?shop=${shop}`;
        break;
      case 'LIMIT_EXCEEDED':
        // Show upgrade prompt
        showUpgradeModal();
        break;
      case 'RATE_LIMIT_EXCEEDED':
        // Wait and retry
        await delay(1000);
        return retry();
      default:
        // Show generic error
        showError(error.message);
    }
  }

  const data = await response.json();
  return data;
} catch (err) {
  console.error('API call failed:', err);
  showError('Network error. Please try again.');
}
```

API Versioning

Current Version: v1 (implicit)

Future Versions:
- v2 endpoints will use `/api/v2/...` prefix
- v1 will be maintained for at least 6 months after v2 release
- Deprecation notices sent 3 months in advance

Rate Limiting Details

Rate Limit Headers

```
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 95
X-RateLimit-Reset: 1672531200
```

Rate Limit Response

```
{
  "error": "Rate limit exceeded",
  "code": "RATE_LIMIT_EXCEEDED",
  "retryAfter": 60,
  "limit": 100,
  "remaining": 0,
  "reset": 1672531200
}
```

Pagination (Future)

Not yet implemented. Future list endpoints will use:

```
GET /api/qryx/conversations?page=2&limit=50
```

Response:

```
{
  "data": [...],
  "pagination": {
    "page": 2,
    "limit": 50,
    "total": 500,
    "pages": 10,
    "hasNext": true,
    "hasPrev": true
  }
}
```

Webhooks Summary

Webhook	URL	Events	Authentication
Stripe	`/api/stripe/webhook`	check-out.session.completed, custom-er.subscription., in-voice.	Stripe Signature
Clerk	`/api/webhooks/clerk`	user., organization.	Svix Signature
Shopify	`/api/webhooks/shopi-fy`	(Future: orders/cre-ate, etc.)	HMAC Signature

Testing APIs

Using cURL

```
# Health check
curl https://www.jnxlabs.ai/api/system/health

# Get config (authenticated)
curl -H "Authorization: Bearer <clerk_token>" \
  https://www.jnxlabs.ai/api/qryx/config?shop=shopbotv3.myshopify.com

# Send chat message
curl -X POST https://www.jnxlabs.ai/api/qryx/chat \
  -H "Content-Type: application/json" \
  -d '{
    "shop": "shopbotv3.myshopify.com",
    "message": "Hello",
    "conversationId": "test-123"
  }'
```

Using Postman

1. Import collection (future: provide Postman collection JSON)
2. Set environment variables:
   - `baseUrl`: `https://www.jnxlabs.ai`
   - `clerkToken`: `<your_clerk_token>`
   - `shop`: `shopbotv3.myshopify.com`
3. Run requests

Summary

This reference covers:

✅ **13 API Endpoints**: Complete request/response documentation
✅ **Authentication**: Clerk, Stripe, Shopify methods
✅ **Error Handling**: Standard codes and best practices
✅ **Webhooks**: Stripe & Clerk integration
✅ **Rate Limiting**: Limits and headers
✅ **Testing**: cURL examples

Related Documentation:

- Stripe Setup Guide (/STRIPE_SETUP_GUIDE.md)
- Testing Guide (/TESTING_GUIDE_PHASE5A.md)
- Troubleshooting Guide (/TROUBLESHOOTING_GUIDE.md)
- Database Schema Reference (/DATABASE_SCHEMA_REFERENCE.md)

Document Version: 1.0
Last Updated: December 29, 2025
Maintained By: JNXLabs Engineering Team